# CLOUD COMPUTING & DEVOPS PRACTICAL EXAM

**Lab Assignment: Multi-Tier Flask + MySQL Web Application Deployment**

| | | |
|---|---|---|
| **Subject:** Cloud Application Deployment | **Time Allowed:** 2 Hours (120 Mins) | **Total Marks:** 100 Marks |
| **Target Stack:** Python (Flask), PyMySQL, MySQL Server | **Deployment Target:** AWS EC2 / Azure VM / GCP / Local Host | **Evaluation Mode:** Practical Demonstration |

## 1. Objective & Scenario

**Scenario:** You are working as a Cloud DevOps Engineer. A team has delivered a Flask web application that requires database connection pooling with MySQL. Your objective is to set up the MySQL relational database, configure environment variables, secure database credentials, and successfully deploy the web application so that users can register, login, reset passwords, and administrators can monitor user registrations.

## 2. Tasks & Functional Requirements

**Task 1: MySQL Database Initialization (20 Marks)**

- Install and start MySQL Server on your deployment server.
- Create a database named cloud_test_db.
- Create a table named users with columns: id (Auto PK), username (UNIQUE), email (UNIQUE), password_hash, role ('user'/'admin'), and created_at.
- Insert a default administrator account (Username: admin, Password: admin123 hashed).

**Task 2: Flask App & Environment Configuration (20 Marks)**

- Set up a Python Virtual Environment (venv) and install packages from requirements.txt.
- Configure .env file with valid MySQL connection parameters (MYSQL_HOST, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DB).
- Ensure the app successfully establishes a database connection without hardcoding secret keys.

**Task 3: User Authentication & Protected Dashboard (20 Marks)**

- **User Registration:** Allow new users to sign up. Store passwords securely using password hashing.
- **User Login:** Authenticate user credentials and create session cookies.
- **User Dashboard:** Restrict access to authenticated users only; display account profile details.
- **Password Reset:** Allow users to reset their passwords using their username or email.

**Task 4: Administrator Management Portal (20 Marks)**

- Provide a dedicated Admin Login interface (/admin/login).
- Verify that only users with role='admin' can access the Admin Dashboard.
- Display live metrics: Total Registered Student Users and Total Administrator Accounts.
- Render a data table listing all registered users with their username, email, role, and registration date.

**Task 5: Cloud Deployment & Security Configuration (20 Marks)**

- Run the application on host 0.0.0.0 and port 5000.
- Configure inbound firewall rules (AWS Security Group / Azure NSG / ufw) to allow port 5000 traffic.
- Demonstrate a working live web application accessible via IP address or Domain.

## 3. Evaluation Rubric & Marking Scheme

| Criteria | Description | Max Marks |
| --- | --- | --- |
| Database Setup | Correct schema, constraints, and default admin seed query execution | 20 Marks |
| App Configuration | Environment variables set up in .env, packages installed cleanly | 20 Marks |
| User Features | Registration, Login, Password Reset, and Dashboard working properly | 20 Marks |
| Admin Features | Admin authentication, count metrics, and user table working properly | 20 Marks |
| Cloud Deployment | Application live and accessible over server Public IP / URL | 20 Marks |
| **Total Score** | **Practical Demonstration & Code Review** | **100 Marks** |

## 4. Student Submission Deliverables

Each student must present the following during practical evaluation:

1. Live Demonstration URL: http://<your-server-ip>:5000
2. Admin Credentials Demonstration using admin account.
3. MySQL CLI Output showing SELECT * FROM users; with at least 2 registered student users.